ITM SKILLS UNIVERSITY - SCHOOL OF FUTURE TECH

Database Management Systems - B.Tech CSE 2025-29, Semester III

CASE STUDY 131: Spice Grinding Unit Order Fulfilment

Database Using SQL (PostgreSQL) - Focus: Transactions, ACID and Concurrency

Name	Bhaumik Keer	Roll No	150096725030

1. Case Study Overview

A spice grinding unit takes bulk orders for ground spice blends (Garam Masala, Chilli Powder, Turmeric Powder ...), grinds them in **batches** and **dispatches** the packed spice to retailers against customer **orders**. The unit manager needs a database for the day-to-day work and for the reports needed at the close of every production day. Without one, spice that was never ground can be dispatched, more than the customer ordered can be shipped, and two staff editing the same record can overwrite each other. The case study therefore focuses on **transactions, ACID and concurrency**.

What was asked and where it is answered. Four tables (blends, dispatches given; production_batches, orders designed by me) so that every dispatch traces back to its start; nothing in dispatches without a matching blend; a dispatch only from an existing, complete batch; six transaction tasks; a full-scale dataset with filtering, join, GROUP BY/HAVING and a subquery; ER diagram; one screenshot per query.

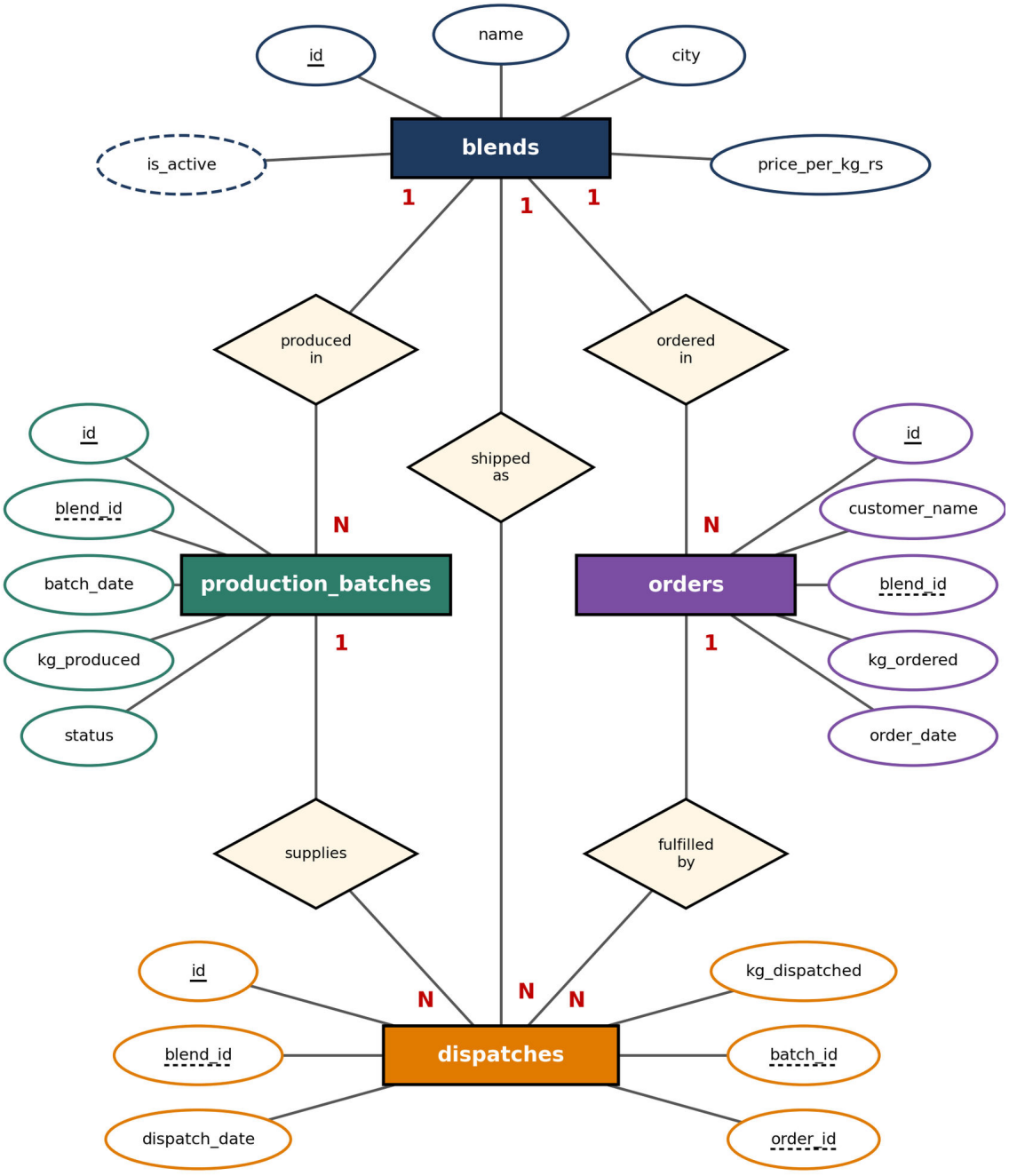
Case-study task	Query	Case-study task	Query
BEGIN ... COMMIT dispatch insert	Q01	New blend + first dispatch (atomic)	Q07
What ROLLBACK undoes	Q02	If the second insert fails	Q08
Lost update (same dispatch)	Q03	Derived is_active needs a trigger	Q09
Fix that closes the lost update	Q04	Filtering / Join	Q10 / Q11
SAVEPOINT	Q05	GROUP BY-HAVING / Correlated subquery	Q12 / Q13
Isolation level + justification	Q06	ACID explained with evidence	Section 5

2. Database Design

Four entities with their own identity and facts are tracked, so there are four tables. Storing each fact once avoids the update anomalies of a single big table (blend name and price would repeat on every dispatch and could disagree).

Table	Purpose	Primary key	Foreign keys and important columns
blends	Master list of spice blends and price per kg	id (SERIAL)	name VARCHAR UNIQUE NOT NULL; city; price_per_kg_rs NUMERIC(8,2) CHECK > 0; is_active BOOLEAN (derived, kept by trigger)
production_batches	One grinding run: source of stock	id (SERIAL)	blend_id -> blends; batch_date DATE; kg_produced INT CHECK > 0; status COMPLETE / IN_PROGRESS; UNIQUE (id, blend_id)
orders	Customer demand: limit for shipping	id (SERIAL)	customer_name; blend_id -> blends; kg_ordered INT CHECK > 0; order_date DATE; UNIQUE (id, blend_id)
dispatches	Daily log; junction between batches and orders	id (SERIAL)	blend_id -> blends; (batch_id, blend_id) -> batches; (order_id, blend_id) -> orders; kg_dispatched INT CHECK > 0; dispatch_date DATE. All NOT NULL

2.1 ER diagram



Bhaumik Keer | Roll No 150096725030

Figure 1 - ER diagram. Primary keys are underlined (solid); foreign keys are underlined with a dashed line; is_active is a derived attribute (dashed ellipse). All relationships are one-to-many (1 to N).

dispatches is the junction table: between batches and orders there is really a many-to-many relationship (one order can be filled from several batches, one batch serves several orders). It is resolved by dispatches, which holds a foreign key to each side plus its own facts (kg_dispatched, dispatch_date).

2.2 Why these keys, types and constraints

- **Primary keys.** Every table has an integer surrogate key *id* (SERIAL): short, never changes, auto-generated, fast to join, and it matches the id column in the sample data. Natural keys were rejected: names can be renamed, customer_name repeats across orders, and (blend, date) is not unique because two dispatches of one blend can happen on the same day. The natural candidate key is still protected with UNIQUE (name).
- **Foreign keys and NOT NULL.** blend_id, batch_id and order_id in dispatches are NOT NULL foreign keys, so a dispatch cannot exist without a blend, batch and order. ON DELETE RESTRICT stops a blend with history from being deleted (CASCADE would silently wipe records).
- **Composite foreign keys.** dispatches references (batch_id, blend_id) and (order_id, blend_id). A dispatch is accepted only if that exact pair exists, so a Chilli Powder dispatch can never use a Garam Masala batch or order. This is why batches and orders carry UNIQUE (id, blend_id), which PostgreSQL requires for the reference.
- **Data types.** NUMERIC(8,2) for money because FLOAT is inexact; INT for whole kilograms as in the sample data; DATE because only dates are recorded; BOOLEAN for is_active.
- **CHECK constraints** reject zero or negative prices and quantities and invalid status text. **Indexes** on dispatches (blend_id, batch_id, order_id, dispatch_date) speed up joins, trigger sums and date filters, because PostgreSQL does not index foreign keys automatically.
- **Normalisation.** 1NF (atomic values), 2NF (all keys are single columns) and 3NF (blend facts only in blends, batch facts only in batches, customer_name depends only on the order). The one deliberate exception is the derived is_active flag, which is why it needs a trigger (Q09).

2.3 Triggers (rules a CHECK constraint cannot express)

- **check_dispatch** (BEFORE INSERT/UPDATE on dispatches): locks the batch row with SELECT ... FOR UPDATE so two staff cannot take the last kilos together; rejects a batch that is not COMPLETE; rejects a dispatch date before the batch or order date; rejects a total dispatched from the batch above kg_produced or for the order above kg_ordered.
- **refresh_is_active** (AFTER INSERT/UPDATE/DELETE on dispatches): recomputes blends.is_active (TRUE when the blend has at least one dispatch) inside the same transaction, so it is undone by a rollback.

2.4 Sample data

The seed data has 12 blends, 18 batches, 31 orders and 29 dispatches (tens of rows) and includes all the sample rows of the case study. Dispatches are loaded last, after the triggers exist, so every seed row is itself validated. **One correction:** sample dispatch 1 was dated 2024-01-08, before batch 1 existed (2024-01-12), which breaks the rule that a dispatch needs an existing batch; its date was changed to 2024-01-15. Every dispatch stores its blend, batch and order, so any rupee value (kg x price) traces back to where it started.

3. Database Creation, Triggers and Data Loading

This section shows the psql session in which the database was created: the database and its four tables with indexes (Figure 3.1), the two triggers (Figures 3.2 and 3.3) and the seed data loaded in dependency order - blends, then batches, then orders, then dispatches (Figures 3.4 to 3.7). The same statements are in **project.sql**. Loading dispatches last means the trigger validated all 29 rows, and INSERT 0 29 shows that none was refused.

```
bhaumikkkeer@Bhaumeekks-MacBook-Air ~ % psql postgres
psql (18.4 (Homebrew))
Type "help" for help.

postgres=# CREATE DATABASE spice_db;
CREATE DATABASE
postgres=# \c spice_db
You are now connected to database "spice_db" as user "bhaumikkkeer".
spice_db=# -- Case Study 131 : Spice Grinding Unit
spice_db=# -- Student : Bhaumik Keer   Roll No : 150096725030
spice_db=# CREATE TABLE blends (
    id               SERIAL        PRIMARY KEY,
    name             VARCHAR(60)   NOT NULL UNIQUE,
    city             VARCHAR(40)   NOT NULL,
    price_per_kg_rs  NUMERIC(8,2)  NOT NULL CHECK (price_per_kg_rs > 0),
    is_active        BOOLEAN       NOT NULL DEFAULT FALSE
);
CREATE TABLE
spice_db=# CREATE TABLE production_batches (
    id           SERIAL       PRIMARY KEY,
    blend_id     INT          NOT NULL REFERENCES blends(id) ON DELETE RESTRICT,
    batch_date   DATE         NOT NULL,
    kg_produced  INT          NOT NULL CHECK (kg_produced > 0),
    status       VARCHAR(12)  NOT NULL DEFAULT 'COMPLETE' CHECK (status IN ('IN_PROGRESS','COMPLETE')),
    UNIQUE (id, blend_id)
);
CREATE TABLE
spice_db=# CREATE TABLE orders (
    id             SERIAL       PRIMARY KEY,
    customer_name  VARCHAR(60)  NOT NULL,
    blend_id       INT          NOT NULL REFERENCES blends(id) ON DELETE RESTRICT,
    kg_ordered     INT          NOT NULL CHECK (kg_ordered > 0),
    order_date     DATE         NOT NULL,
    UNIQUE (id, blend_id)
);
CREATE TABLE
spice_db=# CREATE TABLE dispatches (
    id             SERIAL  PRIMARY KEY,
    blend_id       INT     NOT NULL REFERENCES blends(id) ON DELETE RESTRICT,
    kg_dispatched  INT     NOT NULL CHECK (kg_dispatched > 0),
    dispatch_date  DATE    NOT NULL,
    batch_id       INT     NOT NULL,
    order_id       INT     NOT NULL,
    FOREIGN KEY (batch_id, blend_id) REFERENCES production_batches (id, blend_id),
    FOREIGN KEY (order_id, blend_id) REFERENCES orders (id, blend_id)
);
CREATE TABLE
spice_db=# CREATE INDEX idx_dispatches_blend ON dispatches (blend_id);
CREATE INDEX
spice_db=# CREATE INDEX idx_dispatches_batch ON dispatches (batch_id);
CREATE INDEX
spice_db=# CREATE INDEX idx_dispatches_order ON dispatches (order_id);
CREATE INDEX
spice_db=# CREATE INDEX idx_dispatches_date  ON dispatches (dispatch_date);
CREATE INDEX
```

Figure 3.1 - CREATE DATABASE spice_db, connection with \c, and the four CREATE TABLE statements (blends, production_batches, orders, dispatches) with their primary keys, foreign keys, composite foreign keys and CHECK constraints, followed by the four indexes.

```
spice_db=#
spice_db=# -- Student : Bhaumik Keer   Roll No : 150096725030
spice_db=# -- Trigger 1: business rules that need other rows
spice_db=# CREATE FUNCTION check_dispatch() RETURNS trigger AS $$
spice_db$# DECLARE
    b        production_batches%ROWTYPE;
    o        orders%ROWTYPE;
    used_b   INT;
    used_o   INT;
spice_db$# BEGIN
spice_db$# SELECT * INTO b FROM production_batches WHERE id = NEW.batch_id FOR UPDATE;
spice_db$# IF NOT FOUND THEN RAISE EXCEPTION 'Batch % does not exist', NEW.batch_id; END IF;
spice_db$# SELECT * INTO o FROM orders WHERE id = NEW.order_id;
spice_db$# IF NOT FOUND THEN RAISE EXCEPTION 'Order % does not exist', NEW.order_id; END IF;
spice_db$# IF b.status <> 'COMPLETE' THEN
spice_db$# RAISE EXCEPTION 'Batch % is not complete yet', b.id;
spice_db$# END IF;
spice_db$# IF NEW.dispatch_date < b.batch_date THEN
spice_db$# RAISE EXCEPTION 'Dispatch date % is before batch date %', NEW.dispatch_date, b.batch_date;
spice_db$# END IF;
spice_db$# IF NEW.dispatch_date < o.order_date THEN
spice_db$#  RAISE EXCEPTION 'Dispatch date % is before order date %', NEW.dispatch_date, o.order_date;
spice_db$# END IF;
spice_db$# SELECT COALESCE(SUM(kg_dispatched),0) INTO used_b FROM dispatches
     WHERE batch_id = NEW.batch_id AND id IS DISTINCT FROM NEW.id;
    IF used_b + NEW.kg_dispatched > b.kg_produced THEN
        RAISE EXCEPTION 'Batch % has only % kg left, cannot dispatch % kg',
                        b.id, b.kg_produced - used_b, NEW.kg_dispatched;
    END IF;
spice_db$# SELECT COALESCE(SUM(kg_dispatched),0) INTO used_o FROM dispatches
     WHERE order_id = NEW.order_id AND id IS DISTINCT FROM NEW.id;
    IF used_o + NEW.kg_dispatched > o.kg_ordered THEN
        RAISE EXCEPTION 'Order % has only % kg left to ship, cannot dispatch % kg',
                        o.id, o.kg_ordered - used_o, NEW.kg_dispatched;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE FUNCTION
spice_db=# CREATE TRIGGER trg_check_dispatch
    BEFORE INSERT OR UPDATE ON dispatches
    FOR EACH ROW EXECUTE FUNCTION check_dispatch();
CREATE TRIGGER
spice_db=#
```

Figure 3.2 - Trigger 1: function check_dispatch() (batch exists and is COMPLETE, dates in order, kg dispatched never above kg produced or kg ordered, batch row locked with FOR UPDATE) and trigger trg_check_dispatch (BEFORE INSERT OR UPDATE on dispatches).

```
spice_db=#
spice_db=# -- Student : Bhaumik Keer   Roll No : 150096725030
spice_db=# -- Trigger 2: keeps the derived blends.is_active flag correct
spice_db=# CREATE FUNCTION refresh_is_active() RETURNS trigger AS $$
spice_db$# BEGIN
spice_db$# IF TG_OP <> 'DELETE' THEN
spice_db$# UPDATE blends SET is_active = EXISTS (SELECT 1 FROM dispatches WHERE blend_id = NEW.blend_id)
spice_db$# WHERE id = NEW.blend_id;
spice_db$# END IF;
spice_db$# IF TG_OP <> 'INSERT' THEN
spice_db$# UPDATE blends SET is_active = EXISTS (SELECT 1 FROM dispatches WHERE blend_id = OLD.blend_id)
spice_db$# WHERE id = OLD.blend_id;
spice_db$# END IF;
spice_db$# RETURN NULL;
spice_db$# END;
spice_db$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
spice_db=# CREATE TRIGGER trg_refresh_is_active
    AFTER INSERT OR UPDATE OR DELETE ON dispatches
    FOR EACH ROW EXECUTE FUNCTION refresh_is_active();
CREATE TRIGGER
spice_db=#
```

Figure 3.3 - Trigger 2: function refresh_is_active() and trigger trg_refresh_is_active (AFTER INSERT, UPDATE or DELETE on dispatches) keep the derived blends.is_active flag correct.

```
spice_db=# -- ============ SEED DATA ==============
spice_db=# -- Student : Bhaumik Keer   Roll No : 150096725030
spice_db=# INSERT INTO blends (id, name, city, price_per_kg_rs) VALUES
  (1, 'Garam Masala', 'Jaipur', 1005),
  (2, 'Chilli Powder', 'Ahmedabad', 980),
  (3, 'Turmeric Powder', 'Chennai', 455),
  (4, 'Sambar Masala', 'Jaipur', 1080),
  (5, 'Kitchen King Masala', 'Bengaluru', 555),
  (6, 'Pav Bhaji Masala', 'Ahmedabad', 580),
  (7, 'Coriander Powder', 'Pune', 420),
  (8, 'Cumin Powder', 'Unjha', 690),
  (9, 'Chai Masala', 'Kolkata', 640),
  (10, 'Biryani Masala', 'Hyderabad', 890),
  (11, 'Chaat Masala', 'Delhi', 520),
  (12, 'Black Pepper Powder', 'Kochi', 1250);
INSERT 0 12
spice_db=# SELECT * FROM blends;
 id |        name          |   city    | price_per_kg_rs | is_active
----+----------------------+-----------+-----------------+-----------
  1 | Garam Masala         | Jaipur    |         1005.00 | f
  2 | Chilli Powder        | Ahmedabad |          980.00 | f
  3 | Turmeric Powder      | Chennai   |          455.00 | f
  4 | Sambar Masala        | Jaipur    |         1080.00 | f
  5 | Kitchen King Masala  | Bengaluru |          555.00 | f
  6 | Pav Bhaji Masala     | Ahmedabad |          580.00 | f
  7 | Coriander Powder     | Pune      |          420.00 | f
  8 | Cumin Powder         | Unjha     |          690.00 | f
  9 | Chai Masala          | Kolkata   |          640.00 | f
 10 | Biryani Masala       | Hyderabad |          890.00 | f
 11 | Chaat Masala         | Delhi     |          520.00 | f
 12 | Black Pepper Powder  | Kochi     |         1250.00 | f
(12 rows)
```

Figure 3.4 - Seed data for blends: INSERT 0 12 and SELECT showing all 12 blends (the 6 sample blends plus 6 more).

```
spice_db=# -- Student : Bhaumik Keer   Roll No : 150096725030
spice_db=# INSERT INTO production_batches (id, blend_id, batch_date, kg_produced) VALUES
  (1, 1, '2024-01-12', 500),
  (2, 2, '2024-01-20', 350),
  (3, 1, '2024-02-05', 620),
  (4, 3, '2024-02-08', 400),
  (5, 4, '2024-02-25', 300),
  (6, 5, '2024-03-01', 500),
  (7, 6, '2024-03-04', 250),
  (8, 7, '2024-03-10', 450),
  (9, 8, '2024-03-15', 380),
  (10, 9, '2024-03-22', 300),
  (11, 10, '2024-04-02', 420),
  (12, 11, '2024-04-08', 350),
  (13, 12, '2024-04-15', 200),
  (14, 2, '2024-04-18', 600),
  (15, 3, '2024-05-06', 500),
  (16, 5, '2024-05-14', 450),
  (17, 10, '2024-05-20', 300),
  (18, 8, '2024-06-03', 400);
INSERT 0 18
spice_db=# SELECT * FROM production_batches;
 id | blend_id | batch_date | kg_produced |  status
----+----------+------------+-------------+----------
  1 |        1 | 2024-01-12 |         500 | COMPLETE
  2 |        2 | 2024-01-20 |         350 | COMPLETE
  3 |        1 | 2024-02-05 |         620 | COMPLETE
  4 |        3 | 2024-02-08 |         400 | COMPLETE
  5 |        4 | 2024-02-25 |         300 | COMPLETE
  6 |        5 | 2024-03-01 |         500 | COMPLETE
  7 |        6 | 2024-03-04 |         250 | COMPLETE
  8 |        7 | 2024-03-10 |         450 | COMPLETE
  9 |        8 | 2024-03-15 |         380 | COMPLETE
 10 |        9 | 2024-03-22 |         300 | COMPLETE
 11 |       10 | 2024-04-02 |         420 | COMPLETE
 12 |       11 | 2024-04-08 |         350 | COMPLETE
 13 |       12 | 2024-04-15 |         200 | COMPLETE
 14 |        2 | 2024-04-18 |         600 | COMPLETE
 15 |        3 | 2024-05-06 |         500 | COMPLETE
 16 |        5 | 2024-05-14 |         450 | COMPLETE
 17 |       10 | 2024-05-20 |         300 | COMPLETE
 18 |        8 | 2024-06-03 |         400 | COMPLETE
(18 rows)
```

Figure 3.5 - Seed data for production_batches: INSERT 0 18 and SELECT showing the 18 batches, all COMPLETE.

```
spice_db=# -- Student : Bhaumik Keer   Roll No : 150096725030
spice_db=# INSERT INTO orders (id, customer_name, blend_id, kg_ordered, order_date) VALUES
  (1, 'Sri Ram Traders', 1, 300, '2024-01-10'),
  (2, 'Annapurna Condiments', 2, 200, '2024-01-18'),
  (3, 'Sri Ram Traders', 1, 400, '2024-02-01'),
  (4, 'Sri Ram Traders', 1, 250, '2024-01-14'),
  (5, 'Annapurna Condiments', 2, 250, '2024-01-28'),
  (6, 'Mahalaxmi Stores', 3, 200, '2024-02-10'),
  (7, 'Mahalaxmi Stores', 3, 100, '2024-02-20'),
  (8, 'Spice Route Foods', 4, 80, '2024-02-28'),
  (9, 'Annapurna Condiments', 5, 280, '2024-03-05'),
  (10, 'Gujarat Retail Mart', 5, 150, '2024-03-20'),
  (11, 'Gujarat Retail Mart', 7, 200, '2024-03-12'),
  (12, 'Spice Route Foods', 7, 150, '2024-03-25'),
  (13, 'Mahalaxmi Stores', 8, 220, '2024-03-20'),
  (14, 'Sri Ram Traders', 9, 120, '2024-03-28'),
  (15, 'Annapurna Condiments', 10, 300, '2024-04-05'),
  (16, 'Gujarat Retail Mart', 11, 180, '2024-04-10'),
  (17, 'Spice Route Foods', 2, 300, '2024-04-20'),
  (18, 'Mahalaxmi Stores', 3, 250, '2024-05-08'),
  (19, 'Gujarat Retail Mart', 5, 300, '2024-05-16'),
  (20, 'Sri Ram Traders', 10, 100, '2024-05-22'),
  (21, 'Annapurna Condiments', 8, 250, '2024-06-05'),
  (22, 'Spice Route Foods', 1, 150, '2024-05-01'),
  (23, 'Sri Ram Traders', 4, 150, '2024-04-01'),
  (24, 'Mahalaxmi Stores', 7, 100, '2024-04-12'),
  (25, 'Mahalaxmi Stores', 9, 150, '2024-04-20'),
  (26, 'Gujarat Retail Mart', 12, 150, '2024-06-01'),
  (27, 'Sri Ram Traders', 6, 100, '2024-06-02'),
  (28, 'Spice Route Foods', 11, 120, '2024-04-25'),
  (29, 'Annapurna Condiments', 3, 150, '2024-05-20'),
  (30, 'Sri Ram Traders', 5, 100, '2024-06-01'),
  (31, 'Gujarat Retail Mart', 10, 200, '2024-05-25');
INSERT 0 31
spice_db=# SELECT * FROM orders;
 id |    customer_name     | blend_id | kg_ordered | order_date
----+----------------------+----------+------------+------------
  1 | Sri Ram Traders      |        1 |        300 | 2024-01-10
  2 | Annapurna Condiments |        2 |        200 | 2024-01-18
  3 | Sri Ram Traders      |        1 |        400 | 2024-02-01
  4 | Sri Ram Traders      |        1 |        250 | 2024-01-14
  5 | Annapurna Condiments |        2 |        250 | 2024-01-28
  6 | Mahalaxmi Stores     |        3 |        200 | 2024-02-10
  7 | Mahalaxmi Stores     |        3 |        100 | 2024-02-20
  8 | Spice Route Foods    |        4 |         80 | 2024-02-28
  9 | Annapurna Condiments |        5 |        280 | 2024-03-05
 10 | Gujarat Retail Mart  |        5 |        150 | 2024-03-20
 11 | Gujarat Retail Mart  |        7 |        200 | 2024-03-12
 12 | Spice Route Foods    |        7 |        150 | 2024-03-25
 13 | Mahalaxmi Stores     |        8 |        220 | 2024-03-20
 14 | Sri Ram Traders      |        9 |        120 | 2024-03-28
 15 | Annapurna Condiments |       10 |        300 | 2024-04-05
 16 | Gujarat Retail Mart  |       11 |        180 | 2024-04-10
 17 | Spice Route Foods    |        2 |        300 | 2024-04-20
 18 | Mahalaxmi Stores     |        3 |        250 | 2024-05-08
 19 | Gujarat Retail Mart  |        5 |        300 | 2024-05-16
 20 | Sri Ram Traders      |       10 |        100 | 2024-05-22
 21 | Annapurna Condiments |        8 |        250 | 2024-06-05
 22 | Spice Route Foods    |        1 |        150 | 2024-05-01
 23 | Sri Ram Traders      |        4 |        150 | 2024-04-01
 24 | Mahalaxmi Stores     |        7 |        100 | 2024-04-12
 25 | Mahalaxmi Stores     |        9 |        150 | 2024-04-20
 26 | Gujarat Retail Mart  |       12 |        150 | 2024-06-01
 27 | Sri Ram Traders      |        6 |        100 | 2024-06-02
 28 | Spice Route Foods    |       11 |        120 | 2024-04-25
 29 | Annapurna Condiments |        3 |        150 | 2024-05-20
 30 | Sri Ram Traders      |        5 |        100 | 2024-06-01
 31 | Gujarat Retail Mart  |       10 |        200 | 2024-05-25
(31 rows)
```

Figure 3.6 - Seed data for orders: INSERT 0 31 and SELECT showing the 31 customer orders.

```
spice_db=# -- Student : Bhaumik Keer   Roll No : 150096725030
spice_db=# INSERT INTO dispatches (id, blend_id, kg_dispatched, dispatch_date, batch_id, order_id) VALUES
  (1, 1, 239, '2024-01-15', 1, 1),
  (2, 1, 248, '2024-01-19', 1, 4),
  (3, 2, 239, '2024-02-02', 2, 5),
  (4, 3, 194, '2024-02-14', 4, 6),
  (5, 3, 59, '2024-02-27', 4, 7),
  (6, 4, 59, '2024-03-05', 5, 8),
  (7, 5, 266, '2024-03-16', 6, 9),
  (8, 5, 149, '2024-03-28', 6, 10),
  (9, 1, 380, '2024-02-12', 3, 3),
  (10, 7, 180, '2024-03-18', 8, 11),
  (11, 8, 200, '2024-03-27', 9, 13),
  (12, 7, 150, '2024-03-30', 8, 12),
  (13, 9, 120, '2024-04-02', 10, 14),
  (14, 4, 140, '2024-04-05', 5, 23),
  (15, 10, 250, '2024-04-10', 11, 15),
  (16, 11, 180, '2024-04-16', 12, 16),
  (17, 7, 90, '2024-04-18', 8, 24),
  (18, 2, 120, '2024-04-22', 14, 2),
  (19, 9, 150, '2024-04-24', 10, 25),
  (20, 2, 300, '2024-04-25', 14, 17),
  (21, 11, 100, '2024-04-30', 12, 28),
  (22, 1, 150, '2024-05-03', 3, 22),
  (23, 3, 250, '2024-05-12', 15, 18),
  (24, 5, 300, '2024-05-22', 16, 19),
  (25, 3, 150, '2024-05-25', 15, 29),
  (26, 10, 100, '2024-05-28', 17, 20),
  (27, 10, 180, '2024-05-30', 17, 31),
  (28, 5, 90, '2024-06-06', 16, 30),
  (29, 8, 240, '2024-06-10', 18, 21);
INSERT 0 29
spice_db=# SELECT * FROM dispatches;
 id | blend_id | kg_dispatched | dispatch_date | batch_id | order_id
----+----------+---------------+---------------+----------+----------
  1 |        1 |           239 | 2024-01-15    |        1 |        1
  2 |        1 |           248 | 2024-01-19    |        1 |        4
  3 |        2 |           239 | 2024-02-02    |        2 |        5
  4 |        3 |           194 | 2024-02-14    |        4 |        6
  5 |        3 |            59 | 2024-02-27    |        4 |        7
  6 |        4 |            59 | 2024-03-05    |        5 |        8
  7 |        5 |           266 | 2024-03-16    |        6 |        9
  8 |        5 |           149 | 2024-03-28    |        6 |       10
  9 |        1 |           380 | 2024-02-12    |        3 |        3
 10 |        7 |           180 | 2024-03-18    |        8 |       11
 11 |        8 |           200 | 2024-03-27    |        9 |       13
 12 |        7 |           150 | 2024-03-30    |        8 |       12
 13 |        9 |           120 | 2024-04-02    |       10 |       14
 14 |        4 |           140 | 2024-04-05    |        5 |       23
 15 |       10 |           250 | 2024-04-10    |       11 |       15
 16 |       11 |           180 | 2024-04-16    |       12 |       16
 17 |        7 |            90 | 2024-04-18    |        8 |       24
 18 |        2 |           120 | 2024-04-22    |       14 |        2
 19 |        9 |           150 | 2024-04-24    |       10 |       25
 20 |        2 |           300 | 2024-04-25    |       14 |       17
 21 |       11 |           100 | 2024-04-30    |       12 |       28
 22 |        1 |           150 | 2024-05-03    |        3 |       22
 23 |        3 |           250 | 2024-05-12    |       15 |       18
 24 |        5 |           300 | 2024-05-22    |       16 |       19
 25 |        3 |           150 | 2024-05-25    |       15 |       29
 26 |       10 |           100 | 2024-05-28    |       17 |       20
 27 |       10 |           180 | 2024-05-30    |       17 |       31
 28 |        5 |            90 | 2024-06-06    |       16 |       30
 29 |        8 |           240 | 2024-06-10    |       18 |       21
(29 rows)
```

Figure 3.7 - Seed data for dispatches: INSERT 0 29 and SELECT showing the 29 dispatches. Dispatches are loaded last, so every row was checked by trigger check_dispatch and accepted.

4. Queries, Results and Explanation

Each image shows the name comment, the query and its result exactly as printed by psql. The same queries, in the same order, are in **project.sql**. For each query: the requirement it covers, what it does and why it is written that way, and the alternative with its cost.

Q01. Transaction: insert one dispatch (BEGIN ... COMMIT)

Requirement: Task 1 - transaction inserting a dispatch. **Explanation:** BEGIN starts the transaction; the insert (80 kg Pav Bhaji Masala, batch 7, order 27) is checked by the trigger (batch complete, kilos left, dates), read back, and made permanent by COMMIT. Until COMMIT nobody else sees it (Durability after it). **Alternative / cost:** A single autocommit INSERT stores the same row but gives no chance to check and cancel it; cost of the explicit transaction is none, but it must be closed.

```
spice_db=# -- ==== PART : TRANSACTIONS, ACID AND CONCURRENCY ====
spice_db=# -- Student : Bhaumik Keer   Roll No : 150096725030
spice_db=# -- Q01. Transaction: insert one dispatch (BEGIN ... COMMIT)
spice_db=# BEGIN;
spice_db=# INSERT INTO dispatches (blend_id, kg_dispatched, dispatch_date, batch_id, order_id)
spice_db=# VALUES (6, 80, '2024-06-10', 7, 27);
spice_db=# SELECT id, blend_id, kg_dispatched, dispatch_date, batch_id, order_id
spice_db=# FROM dispatches WHERE order_id = 27;
spice_db=# COMMIT;
BEGIN
INSERT 0 1
 id | blend_id | kg_dispatched | dispatch_date | batch_id | order_id
----+----------+---------------+---------------+----------+----------
 30 |        6 |            80 | 2024-06-10    |        7 |       27
(1 row)

COMMIT
```

Q02. ROLLBACK: what it undoes

Requirement: Task 1 - what a ROLLBACK undoes. **Explanation:** Inserts a 15 kg dispatch for order 28, counts dispatches inside the transaction (2), then ROLLBACK and counts again (1). ROLLBACK undoes everything since BEGIN: the inserted row and the trigger's side effects. Only the used serial number is not returned. **Alternative / cost:** COMMIT then DELETE leaves two visible committed changes and a race with other users; ROLLBACK is atomic and cleaner.

```
spice_db=# -- Q02. ROLLBACK: what it undoes
spice_db=# -- Name - Bhaumik Keer Roll No - 150096725030
spice_db=# BEGIN;
spice_db=# INSERT INTO dispatches (blend_id, kg_dispatched, dispatch_date, batch_id, order_id)
spice_db=# VALUES (11, 15, '2024-06-11', 12, 28);
spice_db=# SELECT COUNT(*) AS dispatches_for_order_28_inside_txn FROM dispatches WHERE order_id = 28;
spice_db=# ROLLBACK;
spice_db=# SELECT COUNT(*) AS dispatches_for_order_28_after_rollback FROM dispatches WHERE order_id = 28;
BEGIN
INSERT 0 1
 dispatches_for_order_28_inside_txn
------------------------------------
                                   2
(1 row)

ROLLBACK
 dispatches_for_order_28_after_rollback
----------------------------------------
                                      1
(1 row)
```

Q03. Lost update on dispatch 9 (kg_dispatched = 380)

Requirement: Task 2 - lost update on the same dispatch. **Explanation:** Dispatch 9 has 380 kg. Both staff read 380; A writes 390 (+10); B writes 385 (+5) from its stale read. Final value 385 instead of 395: A's update is lost. Statements are in the exact order two sessions would run them; the last line resets the row. **Alternative / cost:** Two real terminals prove the same but cannot be captured in one screenshot; here the interleaving is simulated by statement order.

```
spice_db=# -- Q03. Lost update on dispatch 9 (kg_dispatched = 380)
spice_db=# -- Name - Bhaumik Keer Roll No - 150096725030
spice_db=# -- Session A: BEGIN; read 380
-- Session B: BEGIN; read 380
SELECT id, kg_dispatched FROM dispatches WHERE id = 9;
-- Session A: 380 + 10
UPDATE dispatches SET kg_dispatched = 390 WHERE id = 9;
-- Session B: 380 + 5, from its old read (then both COMMIT)
UPDATE dispatches SET kg_dispatched = 385 WHERE id = 9;
SELECT id, kg_dispatched AS final_value_lost_update FROM dispatches WHERE id = 9;
UPDATE dispatches SET kg_dispatched = 380 WHERE id = 9;
 id | kg_dispatched
----+---------------
  9 |           380
(1 row)

UPDATE 1
UPDATE 1
 id | final_value_lost_update
----+-------------------------
  9 |                     385
(1 row)

UPDATE 1
```

Q04. Fix for the lost update: FOR UPDATE + relative update

Requirement: Outcomes - the fix that closes the lost update. **Explanation:** Each session runs SELECT ... FOR UPDATE (row lock) and a relative update (kg_dispatched = kg_dispatched + n). B waits for A, reads 390 and adds 5. Final value 395: both changes survive. **Alternative / cost:** REPEATABLE READ would make B fail with a serialization error and need retry code; a version column needs extra logic. FOR UPDATE is simplest; cost is B waiting while A's transaction is open.

```
spice_db=# -- Q04. Fix for the lost update: FOR UPDATE + relative update
spice_db=# -- Name - Bhaumik Keer Roll No - 150096725030
spice_db=# -- Session A
BEGIN;
SELECT kg_dispatched FROM dispatches WHERE id = 9 FOR UPDATE;
UPDATE dispatches SET kg_dispatched = kg_dispatched + 10 WHERE id = 9;
COMMIT;
-- Session B (waits for A, then sees 390)
BEGIN;
SELECT kg_dispatched FROM dispatches WHERE id = 9 FOR UPDATE;
UPDATE dispatches SET kg_dispatched = kg_dispatched + 5 WHERE id = 9;
COMMIT;
SELECT id, kg_dispatched AS final_value_no_update_lost FROM dispatches WHERE id = 9;
UPDATE dispatches SET kg_dispatched = 380 WHERE id = 9;
BEGIN
 kg_dispatched
---------------
           380
(1 row)

UPDATE 1
COMMIT
BEGIN
 kg_dispatched
---------------
           390
(1 row)

UPDATE 1
COMMIT
 id | final_value_no_update_lost
----+----------------------------
  9 |                        395
(1 row)

UPDATE 1
```

Q05. SAVEPOINT inside a multi-step transaction

Requirement: Task 3 - SAVEPOINT in a multi-step transaction. **Explanation:** Step 1 inserts a 50 kg Chilli Powder dispatch; a SAVEPOINT is set; step 2 wrongly multiplies the price by 10 (shown); ROLLBACK TO SAVEPOINT undoes only step 2 (price back to 980.00); COMMIT keeps step 1. **Alternative / cost:** A full ROLLBACK would also lose the valid dispatch; two separate transactions are not atomic. Cost of a savepoint is negligible.

```
spice_db=# -- Q05. SAVEPOINT inside a multi-step transaction
spice_db=# -- Name - Bhaumik Keer Roll No - 150096725030
spice_db=# BEGIN;
INSERT INTO dispatches (blend_id, kg_dispatched, dispatch_date, batch_id, order_id)
VALUES (2, 50, '2024-06-12', 14, 2);
SAVEPOINT after_dispatch;
UPDATE blends SET price_per_kg_rs = price_per_kg_rs * 10 WHERE id = 2;
SELECT id, name, price_per_kg_rs AS price_after_mistake FROM blends WHERE id = 2;
ROLLBACK TO SAVEPOINT after_dispatch;
SELECT id, name, price_per_kg_rs AS price_restored FROM blends WHERE id = 2;
COMMIT;
SELECT id, blend_id, kg_dispatched, dispatch_date FROM dispatches WHERE order_id = 2 ORDER BY id;
BEGIN
INSERT 0 1
SAVEPOINT
UPDATE 1
 id |     name      | price_after_mistake
----+---------------+---------------------
  2 | Chilli Powder |             9800.00
(1 row)

ROLLBACK
 id |     name      | price_restored
----+---------------+----------------
  2 | Chilli Powder |         980.00
(1 row)

COMMIT
 id | blend_id | kg_dispatched | dispatch_date
----+----------+---------------+---------------
 18 |        2 |           120 | 2024-04-22
 32 |        2 |            50 | 2024-06-12
(2 rows)
```

Q06. Isolation level for recording a dispatch: READ COMMITTED

Requirement: Task 4 - isolation level and two-line justification. **Explanation:** BEGIN ISOLATION LEVEL READ COMMITTED, SHOW transaction_isolation, insert, COMMIT. **Justification:** READ COMMITTED prevents dirty reads, and the only shared resource (kilos left in a batch) is locked inside check_dispatch. REPEATABLE READ / SERIALIZABLE would only add retries without extra safety here. **Alternative / cost:** SERIALIZABLE is also correct but may abort valid transactions, so every dispatch screen would need retry handling.

```
spice_db=# -- Q06. Isolation level for recording a dispatch
spice_db=# -- Name - Bhaumik Keer Roll No - 150096725030
spice_db=# BEGIN ISOLATION LEVEL READ COMMITTED;
SHOW transaction_isolation;
INSERT INTO dispatches (blend_id, kg_dispatched, dispatch_date, batch_id, order_id)
VALUES (4, 10, '2024-06-13', 5, 8);
SELECT id, blend_id, kg_dispatched, dispatch_date FROM dispatches WHERE order_id = 8 ORDER BY id;
COMMIT;
BEGIN
 transaction_isolation
-----------------------
 read committed
(1 row)

INSERT 0 1
 id | blend_id | kg_dispatched | dispatch_date
----+----------+---------------+---------------
  6 |        4 |            59 | 2024-03-05
 33 |        4 |            10 | 2024-06-13
(2 rows)

COMMIT
```

Q07. New blend + its first dispatch as one atomic unit (success)

Requirement: Task 5 - new blend and first dispatch as one atomic unit. **Explanation:** One transaction inserts the blend Tandoori Masala, its batch, an order and the first dispatch (batch and order are NOT NULL foreign keys, so they belong to the unit). currval() reuses the ids just generated. After COMMIT all four rows exist and is_active = t. **Alternative / cost:** Four separate autocommit statements could leave a half-built chain if one failed; hand-typed ids risk mismatches.

```
spice_db=# -- Q07. New blend + its first dispatch as one atomic unit
spice_db=# -- Name - Bhaumik Keer Roll No - 150096725030
spice_db=# BEGIN;
INSERT INTO blends (name, city, price_per_kg_rs) VALUES ('Tandoori Masala', 'Delhi', 760);
INSERT INTO production_batches (blend_id, batch_date, kg_produced)
VALUES (currval('blends_id_seq'), '2024-06-14', 200);
INSERT INTO orders (customer_name, blend_id, kg_ordered, order_date)
VALUES ('Spice Route Foods', currval('blends_id_seq'), 120, '2024-06-14');
INSERT INTO dispatches (blend_id, kg_dispatched, dispatch_date, batch_id, order_id)
VALUES (currval('blends_id_seq'), 100, '2024-06-15',
        currval('production_batches_id_seq'), currval('orders_id_seq'));
COMMIT;
SELECT b.id, b.name, b.is_active, d.kg_dispatched, d.dispatch_date
FROM blends b JOIN dispatches d ON d.blend_id = b.id WHERE b.name = 'Tandoori Masala';

BEGIN
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
COMMIT
 id |      name       | is_active | kg_dispatched | dispatch_date
----+-----------------+-----------+---------------+---------------
 13 | Tandoori Masala | t         |           100 | 2024-06-15
(1 row)
```

Q08. Same atomic unit when the second insert fails (expected ERROR)

Requirement: Task 5 - what happens if the second insert fails. **Explanation:** Same unit for Peri Peri Masala, but the dispatch asks 150 kg from a 100 kg batch. The trigger raises an ERROR (intended); the transaction is aborted, so COMMIT rolls back everything and 0 rows of the blend remain: atomicity - all or nothing. **Alternative / cost:** Cleaning up in application code could be forgotten or fail; the database rollback needs no clean-up code.

```
spice_db=# -- Q08. Same atomic unit when the second insert fails
spice_db=# -- Name - Bhaumik Keer Roll No - 150096725030
spice_db=# BEGIN;
INSERT INTO blends (name, city, price_per_kg_rs) VALUES ('Peri Peri Masala', 'Mumbai', 640);
INSERT INTO production_batches (blend_id, batch_date, kg_produced)
VALUES (currval('blends_id_seq'), '2024-06-14', 100);
INSERT INTO orders (customer_name, blend_id, kg_ordered, order_date)
VALUES ('Spice Route Foods', currval('blends_id_seq'), 200, '2024-06-14');
INSERT INTO dispatches (blend_id, kg_dispatched, dispatch_date, batch_id, order_id)
VALUES (currval('blends_id_seq'), 150, '2024-06-15',
        currval('production_batches_id_seq'), currval('orders_id_seq'));
COMMIT;
SELECT COUNT(*) AS peri_peri_rows_left FROM blends WHERE name = 'Peri Peri Masala';
BEGIN
INSERT 0 1
INSERT 0 1
INSERT 0 1
ERROR:  Batch 20 has only 100 kg left, cannot dispatch 150 kg
CONTEXT:  PL/pgSQL function check_dispatch() line 24 at RAISE
ROLLBACK
 peri_peri_rows_left
---------------------
                   0
(1 row)
```

Q09. Derived is_active flag maintained by a trigger

Requirement: Task 6 - why a derived is_active flag needs a trigger. **Explanation:** is_active for blend 12 goes f -> t when a dispatch is inserted and back to f after ROLLBACK. It is derived from dispatches, so it becomes wrong whenever a dispatch is inserted, updated or deleted unless something recomputes it; the trigger does this automatically, inside the same transaction. **Alternative / cost:** Not storing it (NOT EXISTS or a view) is simpler and never stale; doing it in application code depends on every program remembering. Cost of the trigger: one extra write per change.

```
spice_db=# -- Q09. Derived is_active flag maintained by a trigger
spice_db=# -- Name - Bhaumik Keer Roll No - 150096725030
spice_db=# SELECT id, name, is_active AS before_insert FROM blends WHERE id = 12;
BEGIN;
INSERT INTO dispatches (blend_id, kg_dispatched, dispatch_date, batch_id, order_id)
VALUES (12, 100, '2024-06-16', 13, 26);
SELECT id, name, is_active AS after_insert_inside_txn FROM blends WHERE id = 12;
ROLLBACK;
SELECT id, name, is_active AS after_rollback FROM blends WHERE id = 12;
 id |        name         | before_insert
----+---------------------+---------------
 12 | Black Pepper Powder | f
(1 row)

BEGIN
INSERT 0 1
 id |        name         | after_insert_inside_txn
----+---------------------+-------------------------
 12 | Black Pepper Powder | t
(1 row)

ROLLBACK
 id |        name         | after_rollback
----+---------------------+----------------
 12 | Black Pepper Powder | f
(1 row)
```

Q10. Filtering: dispatches of 150 kg or more in March-April 2024

Requirement: Full dataset - filtering. **Explanation:** Dispatches between 1 March and 30 April 2024 of at least 150 kg. WHERE with BETWEEN and AND states the period and size directly; BETWEEN includes both end dates, and the index on dispatch_date can be used. **Alternative / cost:** >= and <= gives the same result; filtering in the application would transfer needless rows.

```
spice_db=# -- ==== PART : REPORTING QUERIES ON THE FULL DATASET ====
spice_db=# -- Q10. Filtering: dispatches of 150 kg or more in March-April 2024
spice_db=# -- Name - Bhaumik Keer Roll No - 150096725030
spice_db=# SELECT id, blend_id, kg_dispatched, dispatch_date
FROM dispatches
WHERE dispatch_date BETWEEN '2024-03-01' AND '2024-04-30'
  AND kg_dispatched >= 150
ORDER BY dispatch_date;
 id | blend_id | kg_dispatched | dispatch_date
----+----------+---------------+---------------
  7 |        5 |           266 | 2024-03-16
 10 |        7 |           180 | 2024-03-18
 11 |        8 |           200 | 2024-03-27
 12 |        7 |           150 | 2024-03-30
 15 |       10 |           250 | 2024-04-10
 16 |       11 |           180 | 2024-04-16
 19 |        9 |           150 | 2024-04-24
 20 |        2 |           300 | 2024-04-25
(8 rows)
```

Q11. Join: dispatch with blend, customer and rupee value

Requirement: Full dataset - a join. **Explanation:** Joins dispatches to blends (name, price) and orders (customer) and computes value = kg x price. INNER JOIN is right because NOT NULL foreign keys guarantee a match for every dispatch; the result shows traceability from rupee value back to blend and order. **Alternative / cost:** LEFT JOIN would give the same rows and wrongly suggest unmatched dispatches can exist; subqueries in the SELECT list run per row and are harder to read.

```
spice_db=# -- Q11. Join: dispatch with blend, customer and rupee value
spice_db=# -- Name - Bhaumik Keer Roll No - 150096725030
spice_db=# SELECT d.id AS dispatch_id, d.dispatch_date, b.name AS blend, o.customer_name,
       d.kg_dispatched, b.price_per_kg_rs, d.kg_dispatched * b.price_per_kg_rs AS value_rs
FROM dispatches d
JOIN blends b ON b.id = d.blend_id
JOIN orders o ON o.id = d.order_id
ORDER BY d.dispatch_date, d.id
LIMIT 15;
 dispatch_id | dispatch_date |       blend        |    customer_name     | kg_dispatched | price_per_kg_rs | value_rs
-------------+---------------+--------------------+----------------------+---------------+-----------------+-----------
           1 | 2024-01-15    | Garam Masala       | Sri Ram Traders      |           239 |         1005.00 | 240195.00
           2 | 2024-01-19    | Garam Masala       | Sri Ram Traders      |           248 |         1005.00 | 249240.00
           3 | 2024-02-02    | Chilli Powder      | Annapurna Condiments |           239 |          980.00 | 234220.00
           9 | 2024-02-12    | Garam Masala       | Sri Ram Traders      |           380 |         1005.00 | 381900.00
           4 | 2024-02-14    | Turmeric Powder    | Mahalaxmi Stores     |           194 |          455.00 |  88270.00
           5 | 2024-02-27    | Turmeric Powder    | Mahalaxmi Stores     |            59 |          455.00 |  26845.00
           6 | 2024-03-05    | Sambar Masala      | Spice Route Foods    |            59 |         1080.00 |  63720.00
           7 | 2024-03-16    | Kitchen King Masala | Annapurna Condiments |           266 |          555.00 | 147630.00
          10 | 2024-03-18    | Coriander Powder   | Gujarat Retail Mart  |           180 |          420.00 |  75600.00
          11 | 2024-03-27    | Cumin Powder       | Mahalaxmi Stores     |           200 |          690.00 | 138000.00
           8 | 2024-03-28    | Kitchen King Masala | Gujarat Retail Mart  |           149 |          555.00 |  82695.00
          12 | 2024-03-30    | Coriander Powder   | Spice Route Foods    |           150 |          420.00 |  63000.00
          13 | 2024-04-02    | Chai Masala        | Sri Ram Traders      |           120 |          640.00 |  76800.00
          14 | 2024-04-05    | Sambar Masala      | Sri Ram Traders      |           140 |         1080.00 | 151200.00
          15 | 2024-04-10    | Biryani Masala     | Annapurna Condiments |           250 |          890.00 | 222500.00
(15 rows)
```

Q12. GROUP BY / HAVING: revenue per blend (300 kg or more)

Requirement: Full dataset - GROUP BY / HAVING. **Explanation:** Groups dispatches per blend, counts them, sums kg and revenue, and keeps blends with at least 300 kg using HAVING (WHERE cannot filter a SUM because it runs before grouping). Ordered by revenue; Garam Masala is highest. **Alternative / cost:** A derived-table subquery with an outer WHERE works but needs more code. Cost of GROUP BY: individual dispatch details are no longer shown.

```
spice_db=# -- Q12. GROUP BY / HAVING: revenue per blend (300 kg or more)
spice_db=# -- Name - Bhaumik Keer Roll No - 150096725030
spice_db=# SELECT b.name, b.city, COUNT(d.id) AS dispatch_count,
       SUM(d.kg_dispatched) AS total_kg,
       SUM(d.kg_dispatched * b.price_per_kg_rs) AS revenue_rs
FROM blends b
JOIN dispatches d ON d.blend_id = b.id
GROUP BY b.id, b.name, b.city
HAVING SUM(d.kg_dispatched) >= 300
ORDER BY revenue_rs DESC;
        name         |   city    | dispatch_count | total_kg | revenue_rs
---------------------+-----------+----------------+----------+------------
 Garam Masala        | Jaipur    |              4 |     1017 | 1022085.00
 Chilli Powder       | Ahmedabad |              4 |      709 |  694820.00
 Biryani Masala      | Hyderabad |              3 |      530 |  471700.00
 Kitchen King Masala | Bengaluru |              4 |      805 |  446775.00
 Cumin Powder        | Unjha     |              2 |      440 |  303600.00
 Turmeric Powder     | Chennai   |              4 |      653 |  297115.00
 Coriander Powder    | Pune      |              3 |      420 |  176400.00
(7 rows)
```

Q13. Correlated subquery: dispatches above their own blend's average

Requirement: Full dataset - correlated subquery. **Explanation:** For each dispatch the inner query computes the average kg of the same blend (d2.blend_id = d.blend_id); the outer query keeps dispatches above that average. It reads like the question asked, and the index on blend_id makes the inner lookup cheap. **Alternative / cost:** Joining to a GROUP BY average table reads the data once and is faster on very large tables; correlated was chosen for clarity, at the cost of running per row.

```
spice_db=# -- Q13. Correlated subquery: dispatches above their own blend's average
spice_db=# -- Name - Bhaumik Keer Roll No - 150096725030
spice_db=# SELECT d.id, b.name, d.kg_dispatched, d.dispatch_date
FROM dispatches d
JOIN blends b ON b.id = d.blend_id
WHERE d.kg_dispatched > (SELECT AVG(d2.kg_dispatched)
                         FROM dispatches d2
                         WHERE d2.blend_id = d.blend_id)
ORDER BY b.name, d.kg_dispatched DESC;
 id |        name         | kg_dispatched | dispatch_date
----+---------------------+---------------+---------------
 15 | Biryani Masala      |           250 | 2024-04-10
 27 | Biryani Masala      |           180 | 2024-05-30
 16 | Chaat Masala        |           180 | 2024-04-16
 19 | Chai Masala         |           150 | 2024-04-24
 20 | Chilli Powder       |           300 | 2024-04-25
  3 | Chilli Powder       |           239 | 2024-02-02
 10 | Coriander Powder    |           180 | 2024-03-18
 12 | Coriander Powder    |           150 | 2024-03-30
 29 | Cumin Powder        |           240 | 2024-06-10
  9 | Garam Masala        |           380 | 2024-02-12
 24 | Kitchen King Masala |           300 | 2024-05-22
  7 | Kitchen King Masala |           266 | 2024-03-16
 14 | Sambar Masala       |           140 | 2024-04-05
 23 | Turmeric Powder     |           250 | 2024-05-12
  4 | Turmeric Powder     |           194 | 2024-02-14
(15 rows)
```

5. ACID Explained with Evidence from this Database

Property	Evidence in my database	Query
Atomicity	Four inserts (blend, batch, order, dispatch) run in one transaction; the fourth asks for 150 kg from a 100 kg batch and is refused. The COMMIT then acts as a rollback and 0 rows of the new blend remain - the blend from step 1 was undone too. In Q02 a ROLLBACK removes an inserted dispatch (count 2 inside, 1 after).	Q02, Q07, Q08
Consistency	Bad data is refused: over-stock (trigger, Q08), unknown or wrong-blend batch/order (composite foreign keys), negative kg (CHECK), dispatch before batch (trigger). The derived is_active flag always matches the data (Q09).	Q08, Q09
Isolation	Without protection, two staff who both read 380 kg and write 390 and 385 leave 385: one update lost (Q03). With FOR UPDATE and a relative update the second staff waits, reads 390 and the result is 395 (Q04). READ COMMITTED is chosen because the shared resource is locked explicitly (Q06).	Q03, Q04, Q06
Durability	After COMMIT (Q01) the dispatch stays permanently. PostgreSQL confirms a COMMIT only after writing it to the write-ahead log (WAL), which it replays after a crash.	Q01

Lost update, step by step (dispatch 9 = 380 kg): A reads 380; B reads 380; A writes 380 + 10 = 390; B writes 380 + 5 = 385 from its old read. Final 385 instead of 395. **Fix:** SELECT ... FOR UPDATE makes B wait and re-read 390, and kg_dispatched = kg_dispatched + 5 applies to the latest value. Final 395.

6. Design Choices Defended

Choice	Alternative	Why mine, and its cost
Four normalised tables	One wide table	No repeated names/prices, rules can be enforced. Cost: joins.
Composite foreign keys	Single FK + application check	The database guarantees same blend everywhere. Cost: extra UNIQUE (id, blend_id).
Trigger with FOR UPDATE	Application check / CHECK constraint	CHECK cannot see other rows; application checks race. Cost: slightly slower insert.
READ COMMITTED	SERIALIZABLE	Enough because the batch row is locked; SERIALIZABLE adds retry errors. Cost: depends on the lock staying in the trigger.
Stored is_active + trigger	Compute with NOT EXISTS / view	Cheap filters, shows the derived-data problem. Cost: extra trigger, could drift if disabled.
Surrogate id keys, NUMERIC price, RESTRICT	Natural keys, FLOAT, CASCADE	Stable keys, exact money, protected history. Cost: extra column, children deleted first.

7. Limitations and Conclusion

- The introduction mentions recipes and raw spice stock, but the objectives ask for four tables, so these are future work: tables *recipes* and *raw_stock*, with batch creation subtracting raw stock in one transaction.
- Quantities are whole kilograms; fractional kilos would need NUMERIC. customer_name is stored in orders as given; a customers table would be added if customers get more attributes.

The database links blends, batches, orders and dispatches so every dispatch traces to one blend, one completed batch and one real order. Primary keys, composite foreign keys, CHECKs and two triggers make the rules the database's own job: stock cannot go negative, orders cannot be over-shipped and a batch of one blend cannot serve another. The transaction queries show all four ACID properties with evidence, the lost update reproduced and closed, and the reporting queries show filtering, a join, GROUP BY/HAVING and a correlated subquery on the full dataset. Q08 ends with an intended ERROR message - it is the 'second insert fails' case.